Back to the case

STATE OF WORK AND PUBLICATION

A repository for research – one publication for another.

Elif's question

Lea gives Elif access to her research repo. Elif wants to investigate rhyme forms and asks: "Which TEI folder should I take?

Reasonable for Lea's investigation

The repo preserves the acquired sources, the standardized data and the recorded entries of individual studies. The same book is thus available in several well-founded versions. This structure documents Lea's research path. Elif, on the other hand, needs a clearly identified corpus stock for a new study.

REQUESTS

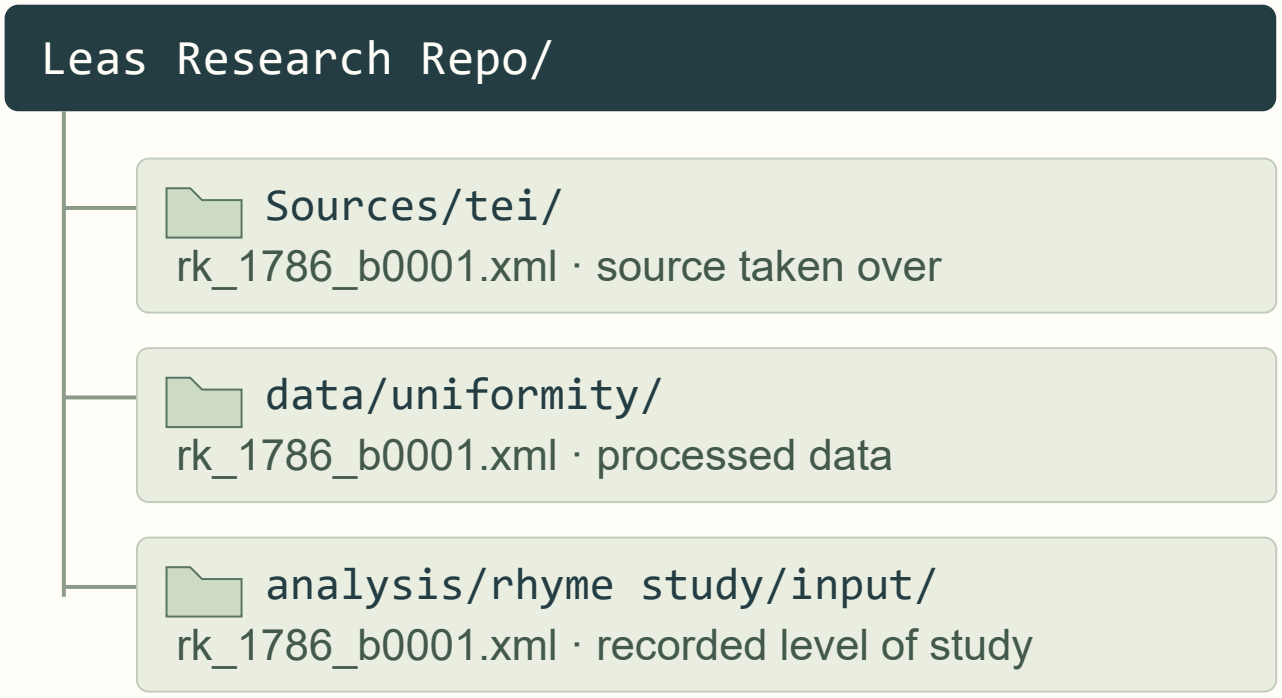
A publication repository is not simply a research repository made public!

A good research repository explains how a study was developed. The data publication explains which inventory others should use for new studies. We set up our own curated corpusrepo for this purpose.

▼ Example and Tip

Example: Research repo: sources/tei/rk_1786_b0001.xml → data/uniformized/ZXq1 → analyses/rhyme study/entry/ZX Q2. Publication repo: exactly one relevant corpus under data/tei / rk_1786_b0001.xml.

Tip: Note publication concept before the first approval in Git.



A research path: Source → Preparation → State of study.

▼ Read graphic as text

Lea's research filing

1. Leas Research Repo/
2. Sources/tei/
3. rk_1786_b0001.xml · source taken over
4. data/uniformity/
5. rk_1786_b0001.xml · processed data
6. analysis/rhyme study/input/
7. rk_1786_b0001.xml · recorded level of study
8. A research path: Source → Preparation → State of study.

TWO PURPOSES · TWO OFFERS

Useful for the study – differently prepared for publication

In the research repo	Why make sense there?	In the publication repository
Sources/tei/rk_1786_b0001.xml	Approved version for comparison with the preparation.	Source and origin in metadata; no competing corpus folder.
data/uniformity/rk_1786_b0001.xml	Prepared version for Leas evaluations.	After verification of the relevant collection under **data/tei/rk_1786_b0001.xml**.
analysis/rhyme study/input/rk_1786_b0001.xml	Fixed input, with which a study result emerged.	The study calls release and selection. Your frozen section is not offered as a second corpus version.

The research repository is appropriate for its purpose. Clear entry, applicable rules and a released inventory are added to the publication.

View the full recommended publication structure ↑

That's how we'd do it.

You can do that.

1. Note publication concept before the first approval in Git.
2. Apply audited book files from data/uniformized to data/tei of the publication repositorys.
3. README, add rules and entry; let the first release be tested externally.

Concrete example

Research repo: sources/tei/rk_1786_b0001.xml → data/uniformized/ZXq1 → analyses/rhyme study/entry/ZX Q2. Publication repo: exactly one relevant corpus under data/tei / rk_1786_b0001.xml.

Done if: A new user recognizes without question which files and which version to examine.

Git helps: Git keeps publication concept, data and corrections together in a comprehensible way. GitHub makes README, releases, issues and audited contributions accessible. The

team decides which work history becomes public.

TEST QUALITY

The script's running.

An unnoticed omission

Lea's new preparation takes over the puzzle texts, but loses the solution part: It was captured in a separate XML element in the previous TEI structure and is no longer included in the newly generated TEI data. Python-Evaluation only counts puzzles and verses: The numbers remain the same. Only when reading a seminar task the answer is missing.

Limiting the error

Lea and Jonas use Git to compare the versions before and after conversion. They discover that the modified converter omits the XML element containing the solutions. They correct the procedure and create a small new regression test that reproduces this specific error: a riddle with a separately encoded solution must pass through the conversion intact. The test then checks that the solution and its link to the riddle have been preserved.

REQUESTS

A successful analysis proves no complete data.

We check structure, relationships and content retention separately. Automatic tests are complemented by a targeted comparison of sources.

▼ Example and Tip

Example: Two puzzles and two solutions before processing; two puzzles and no solution afterwards. XML and Demo-schema exist, the inventory comparison reports the loss.

Tip: Before and after a transformation, compare all elements and content that should be retained, as well as IDs and references.

That's how we'd do it.

You can do that.

1. Before and after a transformation, compare all elements and content that should be retained, as well as IDs and references.
2. Check the diff and affected source points in the pull request; clear unexpected losses.
3. Keep the error as a regression test and publish only after technical approval.

Concrete example

Two puzzles and two solutions before processing; two puzzles and no solution afterwards. XML and Demo-schema exist, the inventory comparison reports the loss.

Done if: The test run detects the known loss; the source control and approval are documented.

Git helps: Version comparison and designated tool stands show when content was lost and which imports are affected. A regression test accompanies the correction. Git preserves changes; the team must check its technical sense.

VERSIONS AND RESEARCH AREA

Yesterday the result was different.

A new graphic

For her publication, Elif calculated the proportion of verse in the corpus and produced a bar chart that seemed particularly revealing. A little later, she wants to use the same script to reproduce the chart in more striking colours. But the bars now tell an unexpected story: Lea has added further collections in the meantime. Elif’s script always loads the current repository version.

Repeat the used version

Using the archived earlier release and the same script, Elif can reproduce her original chart. She needs to identify the data version so that others can check her figures and conclusions. The expanded collection does not change her conclusions, although some figures are now less clear-cut. Elif records the data version, selection of evidence and procedure together. Lea can continue adding books, while earlier studies can still refer back to their specified data version.

REQUESTS

An investigation needs a certain level of data.

We publish audited releases and receive them independently of the Git host. Researchers also call selection, extraction and tooling.

▼ Example and Tip

Example: Demo-v1.0: 2 out of 4 puzzles are verses. Demo-v1.1: 8 out of 10. If the same four old documents are selected, their share remains at 50 percent.

Tip: After the approval create a protected tag and release with inventory and change notes.

That's how we'd do it.

You can do that.

1. After the approval create a protected tag and release with inventory and change notes.
2. Archive data package including metadata and include the concrete version proof in the citation note.
3. For each analysis record release, selection, export rule and script version.

Concrete example

Demo-v1.0: 2 out of 4 puzzles are verses. Demo-v1.1: 8 out of 10. If the same four old documents are selected, their share remains at 50 percent.

Done if: Another person may repeat the examination from the designated data and procedures.

Git helps: Fixed data and tool stands make the comparison repeatable. Git documents inventory changes; the team must explain their scientific significance.

SUPPLEMENTARY USE AND ANNOTATION

A rhyme helps the model to jump.

A question from the seminar

Students ask a local language model to solve rhyming two-line riddles. In one group of these riddles, the solution is also the missing rhyme word in the second line. “Does it understand the riddle, or is it simply completing the rhyme?” To investigate this, the group compares the verse with a separately stored prose adaptation that removes the rhyme clue.

Out of use, follow-up research

Elif adds annotations of form and words to the data used for this study. In particular, she identifies the riddles whose solution is the missing rhyme word. At a later workshop, the prepared data are used to compare different language models. This creates a separate small benchmark corpus: which model can solve verbal riddles in historical varieties of German? Another experiment fine-tunes a language model using the reference data; it achieves substantially better results on the separately held test data. The team decides to prepare the entire existing corpus for such tasks. The data receive an additional annotation layer. TEI allows this layer to be encoded separately and linked to the base data without changing their texts. The extended edition receives a new version number; the base release, annotations and model experiments remain traceably connected.

REQUESTS

New research should be able to build on the corpus.

Our starting point is a separately versioned additional annotation. Originals, processing, automatic layers and model experiments remain clearly distinguishable.

▼ Example and Tip

Example: Base: corpus v1.0. Addition: word annotation 1.0 with reference to its tokenization. Model test: puzzle texts in testadditions.jsonl, reference answers separated in loesungen.jsonll.

Tip: Describe annotation with base release, anchoring or word and generation methods.

That's how we'd do it.

You can do that.

1. Describe annotation with base release, anchoring or word and generation methods.

2. Specify automatic origin and test status; offer basic data without additional layer.
3. In model tests separate test inputs and solutions; test training/test selection and related evidence.

Concrete example

Base: corpus v1.0. Addition: word annotation 1.0 with reference to its tokenization. Model test: puzzle texts in testadditions.jsonl, reference answers separated in loesungen.jsonll.

Done if: The annotation is assigned to its base and can be omitted; the experiment is documented.

Git helps: Basic release, annotation and test scripts can be combined via specified versions. Issues and pull requests enable queries and verified contributions; the versioning makes the investigation comprehensible.

LOCAL USE AND TRANSFER

Does it work without Lea's computer?

An attempt without acclamation

Jonas loads the release to another computer and starts with README. The example still awaits Lea's network drive. Nobody in the team had noticed that: everyone worked with the same infrastructure.

Enhancing entry

Lea replaces personal paths with relative paths and documents calls, dependencies and expected results. Jonas starts again from the fresh package. This time the first examination succeeds without help. Before the end of Lea's job he also tests the instructions for the next release.

REQUESTS

The entry must work without the project environment.

We provide a small full access and the entire designated data set. Another computer is our practical countercheck.

▼ Example and Tip

Example: Instead of a path to Lea's network drive, the script korpus.xml searches relative to its own folder. Entry runs with Python default library without API or account.

Tip: Use filenames without system specific special characters and relative paths.

That's how we'd do it.

You can do that.

1. Use filenames without system specific special characters and relative paths.
2. Document a call with dependencies and expected output in README.
3. Another person can have the newly unpacked package carried out; add contact and care commitment.

Concrete example

Instead of a path to Lea's network drive, the script korpus.xml searches relative to its own folder. Entry runs with Python default library without API or account.

Done if: The documented result is created in a fresh environment; missing knowledge is supplemented in the guide.

Git helps: A designated release, complete local data, versioned examples and clear proofs of change enable independent use. The instructions for later releases are also practically tested.

DOWNLOADS AND ADDITIONAL SERVICES

The library is waiting for the server.

Public and still not usable

Elif wants to compare many puzzles locally. The library loads them individually via the API. The server throttles the requests; their investigation stops.

An Independent Way of Work

The team offers the complete release for download and lets the library also read local files. A small sample leads to the first result. API and frontend remain additional accesses.

REQUESTS

corpus analyses must not depend on the current project server.

Our basic offer is versioned local data and documented exports. We set up an additional service only for a specific need and with secure operation.

▼ Example and Tip

Example: To try out: userstart.zip. For the complete analysis: the designated corpus package. Both describe their files and the respective scope.

Tip: Offer small entry data and complete release separately; share large collection meaningfully.

That's how we'd do it.

You can do that.

1. Offer small entry data and complete release separately; share large collection meaningfully.
2. Library and examples can also be executable with local data.
3. API or Frontend supplement only if benefits, responsibility and operation are clarified.

Concrete example

To try out: userstart.zip. For the complete analysis: the designated corpus package. Both describe their files and the respective scope.

Done if: The investigation can also be continued if the API fails.

Git helps: Git versions data and software. It does not finance or operate any additional services.

PORTABILITY AND CONSERVATION

The host is moving, so I want the research to stay.

Move more than files

When changing the host, the Git files are present. Linked discussions and release attachments are missing. Some decisions are only explained there.

Maintaining the needed context

Jonas takes over the necessary platform information and records professional decisions in the repo. The tests also run outside of the previous automation. Archived releases secure the earlier research references.

REQUESTS

Git history and platform stock are different things.

We secure Git, published packages and the necessary decisions for understanding. The rebuild is checked using a versioned guide.

▼ Example and Tip

Example: Git transfers commits and tags. Release attachments, relevant discussions and service settings are additionally adopted or documented in a comprehensible manner.

Tip: Back up Git references and required additional files; export or document relevant issues and decisions.

That's how we'd do it.

You can do that.

1. Back up Git references and required additional files; export or document relevant issues and decisions.
2. Execute checks and exports without the old CI environment.
3. Check release packages, citations, contact and permissions at the new location.

Concrete example

Git transfers commits and tags. Release attachments, relevant discussions and service settings are additionally adopted or documented in a comprehensible manner.

Done if: The files are understandable at the new location and the documented procedures executable.

Git helps: Git data are portable; scientific archiving and documented procedures limit the dependence on platform functions.